

FRED TALKS

21st May 2026

CONTENTS

Can agents replace the search stack?

DOUG TURNBULL · 1252 WORDS

Six Years Perfecting Maps on watchOS

DAVID SMITH · 1397 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

10 Lessons for Agentic Coding

DREW BREUNIG · 638 WORDS

Can agents replace the search stack?

DOUG TURNBULL · 28 APR 2026 · [SOURCE](#)

How is search implemented where you work? Probably as a complex set of capabilities on top of retrieval. Our search APIs understand queries, call backend search systems, and finally rerank results.

But if we had an agent in the loop, would we need all that? Could we replace search backends with an agent? After all, an agent understands user requests, calls retrieval tools, and evaluates relevance on its own. We see ChatGPT do this all the time, why can't our search bar?

In other words, if you give it a basic BM25 backend to an agent, could it take the Search API's job?

Surprisingly good results with simple tool usage

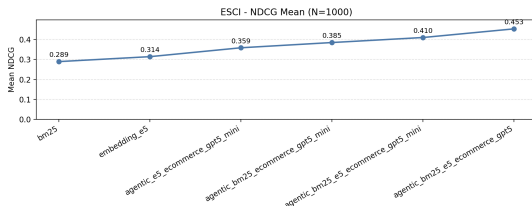
Well I've measured initial findings (a bit deeper than in the past). I gave agents basic search tools and got the popcorn out to watch.

On Amazon ESCI, we see increasing performance with slightly better retrievers + LLMs.

First, we start with a bm25+e5_embedding baseline. They achieve 0.289/0.314 NDCG respectively.

Then we measure three agentic experiments. Each with GPT-5-mini driving different combinations of search tools. We see an agent with e5 embeddings get 0.359 and with BM25 0.385. With both we get 0.410.

GPT-5 with both tools gets an NDCG of 0.453.

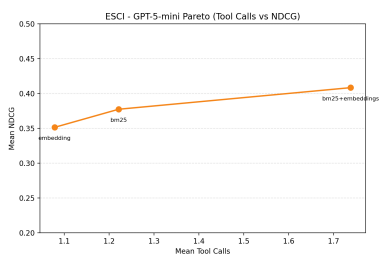


What's amazing about this? I've barely lifted a finger. I just setup a stock LLM with some search tools. Nothing here has been fit to my data. A $0.289 \rightarrow 0.453$ jump in quality depends only on model quality + simple retrievers.

If an agent could be fast enough, is there a future here we build search with agents and not APIs? A world where agents reason about queries to show users the best results?

But, agents call search approximately... once?

What kind of ingenious strategies are these agents employing? It turns out, for the most part, they're calling search roughly once per search tool. You can see this in the graph.



When you look at reasoning traces, you see the agent interpret the search, issue a single query, retrieve up to the max of 20 results, then rank them.

Occasionally, very rarely, with just BM25, you'll see two simultaneous searches. For example, "PVC coupler" shows first a BM25 search for "pvc coupler", the agent, disappointed with results, issues a follow up query (PVC pipe coupler):

Calling `search_bm25` with args `keywords='pvc coupler'` `top_k=20`

```
**Searching for PVC couplers** I noticed that the search results mainly returned RJ45 couplers, which are for network cables, not what the user meant. They're probably asking about PVC pipe couplers used in plumbing. The current search tool isn't yielding entries for PVC pipe couplers specifically. I saw mentions of PVC in descriptions, but it's clearer to use broader terms to refine my search. I need to focus on terms like "PVC pipe coupler" or "coupling" to get better results, so I'll try that next.
```

Calling `search_bm25` with args `keywords='PVC pipe coupler'` `top_k=20`

You also see cases where the agent uses BM25 to issue two queries simultaneously, anticipating inconsistent results:

Calling search_bm25 with args keywords='AN10 oil catch can no filter'
Calling search_bm25 with args keywords='AN10 oil catch can without br

There’s something about keyword search, it seems, that nudges the agent to try more queries. The agent can interpret results, understand why they didn’t match, and take next steps. Keyword search has cause-and-effect (keywords matched / or didn’t). Agents can react to that and improve.

Encouraging exploration improves further

I want agent exploration to be the norm, not the exception.

We haven’t changed a thing about the harness. We could steer agents to search more effectively. What happens with a few small changes to our setup:

- **Require more tool calls** - we ask the agent to keep trying if its made fewer than the required number of tool calls (ie 4). We say something encouraging like “wow these are great, but I bet you can do better!”
- **Require the agent use diverse retrieval queries** - to ensure the agent truly explores, we reject queries too similar to those issued by the agent previously for this search.

Within the GPT-5-mini examples, we see incremental improvements.

strategy	model	mean	median
<u>Two tools</u>	gpt-5-mini	0.4101	0.3743
<u>Require 4 calls, disallow duplicates</u>	gpt-5-mini	0.4290	0.3948
<u>Require 4 calls, disallow queries > 0.9 sim to past queries</u>	gpt-5-mini	0.4308	0.4258

That’s nice, it approaches the GPT-5 benchmark from before, but with a smaller model. Albeit with more tool calls.

What about agentic search models (ie SID-1)?

Another source of hope comes from model providers. Teams like [SID](#) have agentic search models that act as drop-in replacements for your RAG stack. These self-hosted agents translate your company's simple retrieval tools into relevant results. That's basically replacing a search API with an agent.

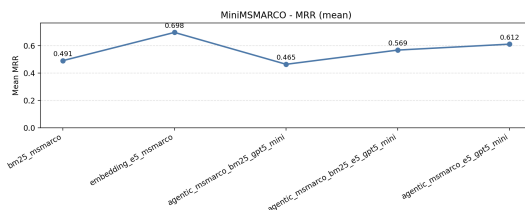
It's important to have a different model. Frontier lab LLMs think of "search" as web search. So they issue a query once, trust the results, and move on. That's valuable because they're reasoning about the user's problem - not search relevance.

But a model like SID-1 might be trained to reason about relevance using less sophisticated retrievers. They know to explore more of the corpus and reason efficiently about results. As a self-hosted subagent, focused only on retrieving results that satisfy the search, they leave the main LLM focused on the user's task.

Still, to be smaller, they must be more focused. One wonders if in the future we'll have agentic search LLMs tailored to different domains. One good at e-commerce. Another good at document retrieval, etc. The tradeoffs look like embedding models: we must choose the model based on its training data. We understand we can drift away from the model's sweet spot.

"Finding" different than Deep Research**

Where does this break down? Change to passage dataset, and there's no improvement by letting an agent drive tools. You can't beat the embedding model.



MiniMARCO MRR

The e5 model here almost certainly has MSMarco in its training set. In this case **the retriever knows best**. The Agents own evaluation of results hits a wall.

We cross a threshold where the LLMs and users diverge in their relevance evaluations. An 80% accurate LLM as a judge isn't that useful, actually. Often that 80% holds the easy cases. It's the last 20% that's hard.

In other words, LLMs don't know everything. Search when the LLM "knows" vs "doesn't know" are different problems.

So far we've focused on finding **things** (products, jobs, etc). But adding knowledge the agent doesn't have? That's fundamentally a different problem. In the case above, the embedding adds knowledge to the agent. When an agent encounters a question it can't answer in that last 20%, it needs search to be an oracle. It's not finding things anymore, it's filling in its own gaps.

That's how ChatGPT treats web search. Ask about a current event past the training cutoff, you'll see it in action.

To me this last 20% says everything about search. It's where the really hard search problems exist. You either:

- Build a model that learns your domain - ie a SID-1 for your data + queries
- Build a thicker / smarter retrieval function, as in classic search. It continues to solve search the old fashioned way - with new techniques like late interaction helping

So in the end, can an agent replace a search API? I'd argue there's still a tremendous amount of skill needed to build a useful retrieval function. We can build a harness so strong even grep / BM25 mostly works. But we pass a threshold with diminishing returns.

Enjoy software Doug in training course form!

Starting May 18!



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search. build better RAG and use LLMs in query understanding.

Six Years Perfecting Maps on watchOS

DAVID SMITH · 17 MAY 2026 · [SOURCE](#)



I love going on wilderness adventures. I am rarely happier than when I am far off into the mountains without a soul in sight. As a result, I have spent a lot of time learning how to safely explore and navigate when I'm away from civilization. The most important habit I've found for not getting lost is to be very regular in checking your location as you go, and the best way I've found to do that is to have a map on my wrist.



For more than **six** years I've been working towards creating the best possible mapping experience on the Apple Watch. With yesterday's [launch of Pedometer++ 8](#), I feel like this design journey has reached a meaningful destination. I would contend that Pedometer++'s watchOS mapping support is the absolute best available on the App Store.

So I wanted to walk through the journey it took to get here.

Early Efforts

I have wanted a good map on my wrist since the Apple Watch launched. This wasn't realistically possible until watchOS 6, which brought SwiftUI to the platform and, for the first time, made “*real*” apps possible. But in those early days, the screens were tiny, and the processors slow. I couldn't *quite* get to where I wanted.



This was my very first attempt that shipped in Pedometer++. These maps were generated completely on the server, which involved sending the relevant workout data roundtrip every time I wanted to refresh the display. This system let me validate the idea, but it was never going to be practically useful for navigation or regular use, and could never work offline.

Custom Mapping Engine

I knew that if I wanted to make progress towards this goal, I'd need to work at a lower level, so I got to work building a fully SwiftUI-native map rendering engine. SwiftUI was the only choice because it's all that watchOS supported, and proved to be helpful for putting maps into widgets, which also only support SwiftUI.



In 2021, I got this engine to a place where I could reliably and performantly render a map on watchOS. With it, I can render any tile-based maps and overlay location information on top.

Map Designs

Next came the question of how best to surface data to users. App design on watchOS is a really fun — but frustrating — challenge. You are designing for a relatively tiny screen, which must be operated one-handed. In this case, I want the user to be able to read the map and use it to navigate, while also having access to other workout-related information.

This began a long series of design attempts, most of which (if I’m being honest) were kinda awful.



Bad designs

In the end, I settled on a “modal” approach where the user can switch between a map screen and a metrics screen using a button on the top-left corner.



Modal Maps

This interface provides one context where the user can freely pan/zoom around the map and another where I can use the more standard watchOS tabbed page interface for metrics and controls. I shipped this to Pedometer++, but there was always something that didn’t quite sit right with me about it.

This design felt like a compromise, and not in a good way. I felt that in order to achieve the goal of making the map interactive, I couldn’t have the map be part of any UI structure that involved swipes. As the screens on Apple Watches got larger, it felt less needed in order to give the map enough space to be useful.

So I set about trying alternative designs. **SO** many designs.



Many more designs

For a while, I thought that I needed to find a way to put the metrics at the bottom of the screen. However, that would lead to other problems on longer outings or for workouts that aren't navigation-focused. So I kept iterating and came up with even *more* designs.



Even more designs

All of these designs suffered from the same fundamental issue: they required the app to display only a fixed set of fields at a time.

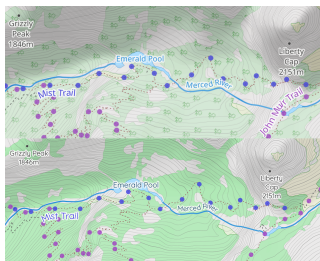
I could make the interface configurable, but one of the *fundamental* rules of watchOS design is that you should avoid any interaction that takes more than a few seconds on the watch. Any user-configurable setup is inherently fiddly, so I didn't like this approach.

Dark Mode, Liquid Glass, & Cartography

Around the same time I was still wrestling with the design challenges of how best to structure the app, Apple announced watchOS 26, and the arrival of [Liquid Glass](#). One of the core design aspects of Liquid Glass is layering stacking elements on top of each other, but another is the types of colors that work best with each other.

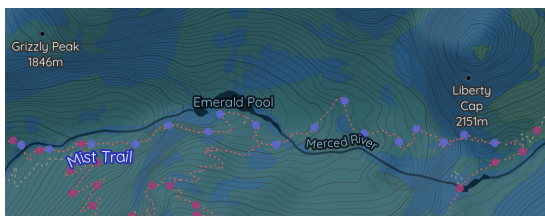
I was previously using [Thunderforest Outdoors](#) as my basemap for the app. I love the content this map includes, but when I started overlaying glassy elements over it I found that it wasn't well-suited for Liquid Glass.

So... I commissioned a custom map. Working with the incredible cartographer [Andy Allen](#), we created a completely new basemap that would look fantastic with Liquid Glass.¹



We simplified the map visually, increased the contrast of the elements, and made the map elements more saturated to prevent them from becoming a muddy mess when shown below glass.

With this work done, I had another opportunity: I could finally have a dark mode variant of the map tiles. While helpful on iOS, this really shines on watchOS. Andy and I really worked toward something which would be incredibly legible at arm's length.

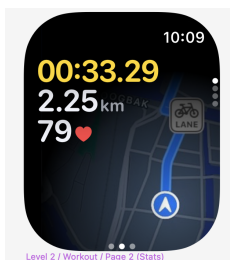


The result of these efforts is that now I have a great map for watchOS... but a design that didn't match that greatness.



Striving for Great

I kept trying. To get me out of my design rut, I enlisted the help of the fantastic designer [Rafa Conde](#). I needed a fresh set of eyes on this and very quickly, this partnership paid off. They proposed a variety of alternative layouts, but when I saw this one I *knew* it was **the one**.



The layering of the metrics on the top-left corner, with the map being the top page of a vertical stack, was the correct answer. This design handles interactivity by requiring a tap on the map first to enter “browse mode”.

Tweaking and Polishing

Now that I had the overall concept locked in, the real fun began, actually building the app and dialing in all the details. I fairly quickly took Rafa’s concept and turned it into a working prototype. This let me validate the idea in the field... literally. After walking a few hundred miles with it, I was confident it was the correct approach.



Next, I needed to dial in the font and make more subtle design choices.



Map fonts

After a bit more iteration, I arrived at the design that shipped yesterday. It is legible, useful, and (in my humble opinion) beautiful.



It feels really good to be able to cap off this six-year journey with a design I couldn't be more proud of. This screen represents so much accumulated effort and learning. It finally gives me a design which feels native on the platform, but also novel and unique.

Here is the evolution of this design over the last six years:



Postscript: Considering MapKit

While my work on watchOS mapping massively predates the arrival of Apple's MapKit onto the platform, it is probably worth explaining why I decided to do all of this custom work to avoid using it.

Fundamentally, I find that MapKit is great for *basic* uses, but doesn't provide nearly the level of configurability and utility which I want Pedometer++ to offer. For example:

- MapKit on watchOS always shows in dark mode, which generally is a good default, but closes the door on some accessibility and user choice reasons. I needed it to be a user-selectable option.
- While MapKit on watchOS has gotten better over time in terms of what you can do with it, I still find it a bit limiting in terms of animations and overlays.

- MapKit's coverage is improving with regards to topographic contours and trail marking, but there are far too many places where the MapKit map is essentially blank, but I know there should be more rich details available. For example, here is my map vs MapKit at the trailhead of one of my favorite hikes in Scotland.



MapKit comparison

1. I still find it so cool that my work on this allows me to say that I “commissioned a cartographer” to work on something for me. 😊 ↩

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson's *[Staff engineer archetypes](#)*. He argues that the “staff engineer” title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the “staff engineer archetypes” and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it's bad practical advice to tell engineers to try and target them.

Archetypes do not make good goals

To see why, let's take the "team lead" archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard "to be a writer". You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - "good work" here means "deliver shareholder value" - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned "minders" (often current staff engineers) who will ghost-lead the project through you².
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The "solver" and "right hand" archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of "aiming for" a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don't get the support needed to succeed) that's their manager's problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can't be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn't possible to conjure useful work from thin air. That's actually by design: **the staff engineer role is supposed to be unfair.** Something many engineers don't realize is that all senior management and executive leadership roles are unfair too, in the same way. That's just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. "Staff engineer" is the first engineering role where you are held largely responsible for outcomes you don't control.

Developing a "staff engineer mindset" thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself "is this useful to the company" (and answering correctly).
2. Lose the habit of worrying about if you're being treated "fairly". Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won't look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You'll also look like someone who's paying a lot of attention to what their manager's actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you'll eventually find yourself in one of the staff engineer archetypes. However, it probably won't be the one you're "aiming for". The whole point of being a staff engineer is that you're willing to fill whatever archetype the company needs at the time.

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the “staff engineer” role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they’ve built a deep relationship of trust with their organization’s leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don’t have the formal “staff” title at all, while others have “staff” as a fairly early rung on the ladder and a panoply of “senior staff”, “senior principal staff”, and so on roles above it. Like all “staff engineer” discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP’s trusted lieutenants can actually be a good way to build trust in the medium-term, but you’d better hope you’ve built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it’s always better to be useful (again, in the sense of “delivering shareholder value”).

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are [here](#) and [here](#).

10 Lessons for Agentic Coding

DREW BREUNIG · 08 MAY 2026 · [SOURCE](#)

What should we do when code is cheap?



Lately, this blog has featured a lot of writing about agentic coding. Frontier models are *really* good at coding these days, much better than they are at other tasks. Coding with agents feels like a preview of the future, a playground for seeing how far we can push agent capabilities. It's invigorating, rewarding, and deeply *weird*.

I've been keeping a running list of tips for agentic coding: guidelines or rules one might give to someone just getting started with Codex, Claude Code, Pi, or any other agent. Ideally each tip is generalizable guidance, relevant to any agentic programming. I'm also looking for durable lessons that will stick around as models and harnesses improve.

Below is my current list: *10 Lessons for Agentic Coding*. Ten's a nice round number; a good time to put this out there.

To be clear: I take credit only for honing and compiling these guidelines. As [Kshetraina Raghavan](#) said to me today, "It's *crazy* how we're all converging on similar lessons."

(If you think I've missed anything below, please reach out!)

10 Lessons for Agentic Coding

1. **Implement to learn.** You can go far with Spec-Driven Development, but the act of writing code surfaces decisions you hadn't considered and makes your spec better. When code is cheap, implement to learn.
2. **Rebuild often.** Implement early and *often* to learn more. Fork and recode crazy thought experiments. Find out how far you can take feature. Of course, you want to iterate and compound your efforts, but cheap code means you can reconnoiter and reinvent in ways you never could.
3. **Invest in end-to-end tests.** When we can reinvent our code cheaply, we should spend time writing tests that measure our product's *functions*, not *how* it performs them. We want behavioral contracts that grant us the freedom to rebuild and reimplement.
4. **Document intent.** Tests detail our goals while code encodes our methods, but neither captures the *why*. Your intent motivates your decisions, and persisting it alongside the code helps you and your agent compound those decisions in a consistent direction.
5. **Keep your specs in sync.** Update your specs, the markdown files containing your goals and plans, as you advance your code and your tests. Treating your spec as a frozen artifact written before work begins, you'll fail to capture learnings during implementation. Keeping it current lets it constantly inform your and your agents' choices, and makes frequent rebuilds easier.
6. **Find the hard stuff.** Work on a project long enough and things will stop being easy. You'll speed through the boilerplate work, the obvious design decisions, and start hitting the ugly, difficult work: intuitive design, performance, security, resilience, and systemic architecture. Anyone can vibe the easy stuff. The hard work is where the value is. Find it and dig in.
7. **Automate everything that's easy.** To spend more time on the hard stuff, minimize the time you spend on easy things. Distill learnings into skills, build loops, automate code reviews, and let your tools compound. But careful: don't get stuck in a Mystery House.
8. **Develop your taste.** When code arrives fast but feedback doesn't, the only source of feedback that keeps up is your own. The better you know your domain, your users, and their problems, the further you can go without checking in.

9. **Agents amplify experience.** Talented developers underestimate how much intuition they bring to their prompts: the right terms, the right framing, and the right level of specificity. If you know your stack, you can save countless cycles during both implementation and debugging, and cut down needless agent exploration. Pair technical expertise coupled with great taste for an unbeatable advantage.
10. **Code is cheap, but maintenance, support, and security aren't.** Agentic code is “free as in puppies.” Support isn't cheap and neither is security. Build fast, but mind the maintenance you're adopting.